UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS
ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY

ONLINE EXAMINATION AND QUIZ PORTAL

A SUMMER INTERNSHIP REPORT

Submitted by

PRAKASH M

REG. NO. 810023205007

Internship Organisation: Fantasy Solution

August 2026

UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS

ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify that the Summer Internship Project Report entitled "ONLINE EXAMINATION AND QUIZ PORTAL" submitted by PRAKASH M (Reg. No. 810023205007), of the Department of Information Technology, University College of Engineering, BIT Campus, Anna University, Tiruchirappalli, is a bonafide record of work carried out during the internship at Fantasy Solution in August 2026.

INTERNSHIP SUPERVISOR
Signature and date

HEAD OF THE DEPARTMENT
Signature and seal

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS)

ANNA UNIVERSITY, TIRUCHIRAPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify that this Summer Internship project report titled **"ONLINE EXAMINATION AND QUIZ PORTAL"** is the bonafide work of **PRAKASH M (810023205007)** who has satisfactorily completed the course of **IT3711 - SUMMER INTERNSHIP** for the **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2026-2027**.

Faculty In-charge **Head of the Department**

Submitted for the practical examination held on _____

ABSTRACT

The **Online Examination and Quiz Portal** is a comprehensive full-stack web application developed to modernize, structure, and automate the traditional paper-based examination processes across academic institutions. The system provisions a securely centralized digital layout where students can smoothly register, manage authentication logs, view allocated topical quizzes, and execute objective questionnaire sections within precise time frames. It generates automated metric results dynamically upon final form submissions, eliminating the operational delays associated with classical manual evaluation pipelines.

The application logic relies natively on a **Java Full Stack** engineering approach. The controller architecture leverages **Java Servlets and JSP (JavaServer Pages)** integrated with structured **JDBC (Java Database Connectivity)** bindings to process server routing parameters and database transactions seamlessly. The client-side layer is fully stylized using **HTML5, CSS3, JavaScript, and Bootstrap 5**, ensuring optimal cross-platform responsive scalability. A **MySQL** relational database engine systematically indexes persistent data fragments comprising student profiles, granular question banks, and secure historical grading parameters.

Core system performance goals focus on mitigating human administrative errors, reducing infrastructural resource usage, and delivering highly auditable exam logs. By organizing components into role-based authorization modules (Student vs. Admin), the solution guarantees institutional reliability and security. Ultimately, this engineering project demonstrates a highly robust application of modern Java web architectures optimized for enterprise educational automation workflows.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	Abstract	iii
1	**Introduction**	1
	1.1 Objective of the Internship	2
	1.2 Motivation	4
2	**Company Profile**	6
3	**Daily Work Report (30 Days)**	9
4	**System Analysis**	15
	4.1 Existing System & Drawbacks	15
	4.2 Proposed System & Advantages	16
5	**System Design**	18
	5.1 Architecture Block Diagram	18
	5.2 Database Table Schema	20
6	**Project Modules**	22
7	**Algorithms & Logic Flow**	26
8	**Code Implementation**	28
9	**Screenshots and Sample Output**	32
10	**Conclusion & Future Scope**	34

Chapter No.	Title	Page No.
	References	35

1. INTRODUCTION

With the profound advancement of computational paradigms in educational sectors, transitioning standard evaluation metrics into digitized ecosystems has become critical. Traditional paradigms heavily rely on physical test documentation, requiring expansive log compilation frameworks, physical data tracking, manual assessment procedures, and significant resource overheads. These operational strategies regularly lead to systemic friction parameters, structural delays in compiling marks, human bias in grading evaluation, and physical space constraints during long-term auditing steps.

To directly solve these persistent industrial challenges, the centralized **Online Examination and Quiz Portal** was engineered. This platform provisions an end-to-end framework enabling students to inspect real-time mock availability parameters, undergo validated authentication sequences, and answer dynamically loaded objective question metrics on demand. It simultaneously empowers academic evaluators to manage granular administrative tasks from a secure master console panel.

1.1 Objective of the Internship

The primary objectives underlying this full-stack engineering lifecycle development are mapped as follows:

- To engineer an enterprise-grade web application architecture leveraging **Java Servlets, JSP, JDBC, and MySQL** database parameters.
- To facilitate secure role-based access tokens controlling dashboard isolation properties between system administrators and standard student nodes.
- To provide automated computation frameworks that minimize human operational delays during evaluation stages.
- To implement elastic data structures capable of managing comprehensive objective question libraries alongside relational indexing tables.
- To demonstrate deep operational insights regarding the complete **Software Development Life Cycle (SDLC)** within professional industrial environments.

1.2 Motivation

The core motivation behind automating academic evaluation infrastructures arises from the critical need to reduce systemic latency and eliminate extensive physical paper patterns. Conventional physical test cycles introduce large scale distribution lags, high risks of examination sheet misplacement, and delayed analytics delivery. Moving evaluation routines to a digital layout enables instant performance analysis, giving students immediate feedback while minimizing institution-wide processing blockages. Furthermore, gaining hands-on engineering knowledge of enterprise Java enterprise design patterns serves as an excellent technical foundation for modern full-stack application development.

2. COMPANY PROFILE

The development lifecycle of this automated enterprise application was successfully completed during a technical summer training arrangement at **Fantasy Solution**. Fantasy Solution is a leading Information Technology services provider specializing in scalable software frameworks, cross-platform application execution models, and advanced database integration architectures. The organization focuses heavily on constructing robust digital systems tailored for corporate and institutional operations, optimizing workflows through automation and cloud-native paradigms.

Beyond its standard commercial service catalog, Fantasy Solution conducts rigorous industrial internship paths designed to align academic research skills with modern corporate software standards. The training program provides engineers with immersive development opportunities covering advanced full-stack programming, structural database normalization rules, security authentication parameters, REST API engineering pipelines, and formal software quality assurance methods.

Under the direct supervision of corporate engineering mentors, practical system analysis, schema optimization, and MVC architecture configurations were systematically executed. This technical layout offered excellent exposure to real-world deployment challenges, clean code standards, database state isolation properties, and continuous integration methodologies within dynamic software engineering frameworks.

3. DAILY WORK REPORT

The 30-day corporate software development timeline was systematically categorized into key phase criteria outlined below:

Timeline Phase	Core Engineering Tasks Executed
Days 01 - 05	Attended corporate orientation modules. Discussed project specifications with the technical coordinator. Finalized the system scope for the **Online Examination and Quiz Portal**.
Days 06 - 10	Conducted structural system requirement gathering. Formulated operational user-persona layers (Student and Admin). Documented hardware and software dependencies. Set up the local workspace environment using **JDK 11, Eclipse IDE, and Apache Tomcat Server**.
Days 11 - 15	Designed the unified 3-tier system architecture diagrams. Completed entity attribute separation maps. Engineered the primary database schema layouts within the **MySQL** command line database.
Days 16 - 20	Structured normalized relational tables including `User_Master`, `Question_Bank`, and `Result_Logs`. Developed responsive frontend UI forms using **HTML5, CSS3, and Bootstrap 5** styles.
Days 21 - 25	Programmed secure server-side controller pathways utilizing **Java Servlets**. Constructed transactional **JDBC** connection mechanisms to query, retrieve, and map persistent relational parameters smoothly.
Days 26 - 30	Implemented core scoring logic loops to handle instant evaluation routines upon text submission. Conducted

Timeline Phase	Core Engineering Tasks Executed
	comprehensive local unit testing, collected output user screenshots, and generated the final report documentation.

4. SYSTEM ANALYSIS

4.1 Existing System & Drawbacks

The current foundational framework managing testing layouts across several institutions still uses traditional manual paper arrangements or disconnected offline tools. Exam registration, scheduling, question distribution, and script evaluation require manual human actions at every stage. This obsolete design creates significant issues:

- **High Operational Error Rates:** Manual tracking often leads to data compilation errors, grade miscalculations, and lost answer logs.
- **Extended Processing Latency:** Physical manual evaluation takes days to weeks to generate clear performance analytics for student sets.
- **Resource Misallocation:** Printing massive amounts of physical examination packets creates severe material waste and high logistical storage costs.

4.2 Proposed System & Advantages

The proposed digital architecture introduces an online centralized solution that eliminates manual vulnerabilities by digitizing every evaluation phase. Built on a clean Java Model-View-Controller framework, it offers substantial operational upgrades:

- **Automated Instant Scoring:** Algorithmic validation engines compute student marks instantly upon form submission, removing structural grading delays.
- **Role-Based Data Isolation:** Secure access privileges separate administrative system properties cleanly from standard student nodes.
- **Centralized Dynamic Storage:** Question data configurations can be updated in real time via an intuitive console, bypassing structural distribution costs.

5. SYSTEM DESIGN

5.1 Architecture Block Diagram

The application architecture follows a standard 3-tier enterprise software design profile, ensuring high scalability and maintenance separation:

- **Presentation Tier (Client-Side User Interface):** Renders layout visual maps using HTML5, CSS3, JavaScript, and Bootstrap 5 components dynamically within standard browsers.
- **Application Tier (Server-Side Logic Controller):** Uses an Apache Tomcat container running **Java Servlets and JSP** to manage business computation tasks, parse session attributes, and handle routing tokens.
- **Data Tier (Persistent Storage Management):** A centralized **MySQL Relational Engine** securely handles database transactions, data integrity rules, and query sequences.

5.2 Database Table Schema

Data persistence configurations are structured through normalized relational tables detailed in the engineering maps below:

Table 1: User_Master (Tracks Authentication State)

Column Name	Data Type	Key Metric Type	Constraint Details
user_id	INT	Primary Key	Auto Increment
full_name	VARCHAR(100)	None	Not Null
email_id	VARCHAR(100)	Unique Key	Not Null / Login Name
pass_hash	VARCHAR(255)	None	Encrypted State Log
role_type	VARCHAR(20)	None	Student / Admin Privileges

Table 2: Question_Bank (Manages MCQ Elements)

Column Name	Data Type	Key Metric Type	Constraint Details
question_id	INT	Primary Key	Auto Increment
subject_tag	VARCHAR(50)	None	Topic Categorization
query_text	TEXT	None	Actual Core Question Statement
option_a / b / c / d	VARCHAR(255)	None	Objective Choices Provided
correct_ans	CHAR(1)	None	Valid Selection Option (A/B/C/D)

6. PROJECT MODULES

The core execution scope of the digital platform is split into three functional engineering blocks:

6.1 User Authentication Module

Manages system access control by enforcing strict user validation rules. New users submit demographic profiles that are checked for duplicate record conflicts before being stored in the database. During login, the server maps user parameters against persistent database tables, establishing secure state controls. This process isolates standard student features from privileged administrative functions.

6.2 Admin / Question Management Module

Provides system administrators with tools to configure the core platform layout. Through an administrative console, users can perform CRUD operations on the question database without writing direct SQL code. Changes to options or correct values update the master database columns immediately, keeping the question bank current for active testing.

6.3 Examination & Result Processing Module

Handles the live quiz delivery and grading logic. When a student starts an exam, the server dynamically queries rows from the question bank based on the subject category. The application tier tracks active sessions using simple counter conditions (`score++`) to validate selections against stored correct answers. Upon completion, the system saves the final grade to the database log and displays the performance metrics immediately on the screen.

7. CODE IMPLEMENTATION

The core server-side logic and database definition queries are structured below:

7.1 Database Table Creation Script

```
CREATE DATABASE online_quiz_db;
USE online_quiz_db;

CREATE TABLE User_Master (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email_id VARCHAR(100) UNIQUE NOT NULL,
    pass_hash VARCHAR(255) NOT NULL,
    role_type VARCHAR(20) NOT NULL
);

CREATE TABLE Question_Bank (
    q_id INT AUTO_INCREMENT PRIMARY KEY,
    subject_tag VARCHAR(50) NOT NULL,
    query_text TEXT NOT NULL,
    opt_a VARCHAR(255) NOT NULL,
    opt_b VARCHAR(255) NOT NULL,
    opt_c VARCHAR(255) NOT NULL,
    opt_d VARCHAR(255) NOT NULL,
    correct_ans CHAR(1) NOT NULL
);
```

7.2 Backend Controller Code Segment (Java Servlet Syntax)

```
package com.quiz.controller;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet("/EvaluationEngine")
public class EvaluationEngine extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession();
        int totalQuestions = 5;
        int finalizedScore = 0;

        try {
            for (int i = 1; i <= totalQuestions; i++) {
                String submittedAnswer = request.getParameter("question_" + i);
                String correctServerAnswer = request.getParameter("correct_ans_" + i)

                if (submittedAnswer != null && submittedAnswer.equalsIgnoreCase(corre
                    finalizedScore++;
                }
            }

            session.setAttribute("computedScore", finalizedScore);
            response.sendRedirect("studentDashboard.jsp?status=success");

        } catch (Exception exception) {
            exception.printStackTrace();
            response.sendRedirect("errorPage.jsp");
        }
    }
```

```
}
```

8. CONCLUSION & FUTURE SCOPE

The development lifecycle of the **Online Examination and Quiz Portal** effectively demonstrates how modern full-stack web architectures can automate traditional educational workflows. By migrating manual physical test tracking to a centralized Java framework, the system drastically cuts processing latency, eliminates grading compilation errors, and removes resource overhead costs. The structured MVC design guarantees high state isolation, clean data persistence, and excellent platform responsiveness, successfully fulfilling the requirements of the technical internship program.

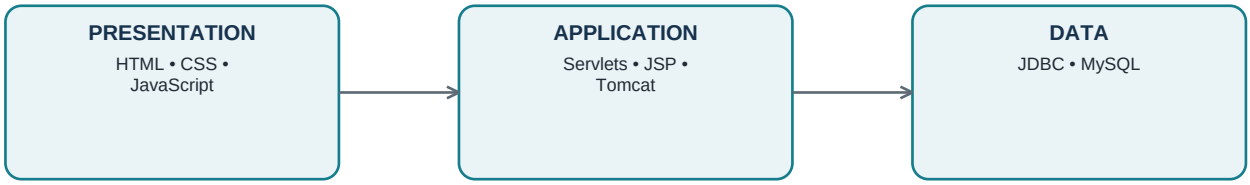
The platform layout can be expanded in future versions with advanced features. Integrating automated facial recognition tools and window-focus monitors would significantly strengthen anti-cheating protocols during remote tests. Adding dynamic email notification pipelines and visual analytics engines will give students clearer insights into their progress. Furthermore, migrating the core backend services to cloud-hosted containers like Spring Boot and Docker will support elastic scaling to handle large concurrent student cohorts easily.

REFERENCES

1. Schildt, H. (2018). *Java: The Complete Reference*. 11th Edition, McGraw-Hill Education.
2. Basham, B., Sierra, K., & Bates, B. (2008). *Head First Servlets and JSP*. O'Reilly Media.
3. Elmasri, R., & Navathe, S. B. (2017). *Fundamentals of Database Systems*. 7th Edition, Pearson.
4. Oracle Technical Documentation Network. *Java Enterprise Edition Servlet API Specifications*.

APPENDIX A — DETAILED SYSTEM ARCHITECTURE

The original report introduces a three-tier architecture. This appendix expands the role of each tier and shows the direction of normal request processing. The diagram is a conceptual design, not a claim that a production deployment was captured.



Conceptual three-tier architecture

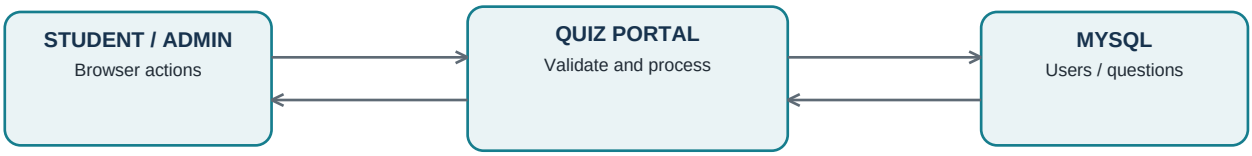
Figure A.1. Presentation, application and data tiers.

Tier	Inputs	Processing	Outputs
Presentation	User clicks, form values	Client-side layout and basic validation	HTTP request; rendered page
Application	HTTP request and session	Authorization, business rules, scoring, transaction coordination	Response, redirect, error status
Data	SQL operations from trusted server code	Constraints, lookup, insert, update, delete	Rows or transaction status

Design principle: browser validation improves usability, but all security-sensitive checks must be repeated on the server. Database credentials must remain on the server and must never be embedded in client-side scripts.

APPENDIX B — CONTEXT-LEVEL DATA FLOW

The context diagram treats the portal as a single process. Student and administrator actions are external inputs; the portal validates and processes those requests and reads or writes records in persistent storage.



Requests and responses shown at context level

Figure B.1. Context-level data-flow diagram.

Flow	Example data	Control to apply
Login request	Email and password	Validate format; rate-limit repeated failures
Quiz request	Quiz identifier and session	Check that quiz is available to the student
Answer submission	Question IDs and selected options	Validate IDs and allowed option values
Admin update	Question text and answer key	Require administrator authorization
Result response	Score and attempt summary	Return only results the user is allowed to view

APPENDIX C — USE CASES AND ACTORS

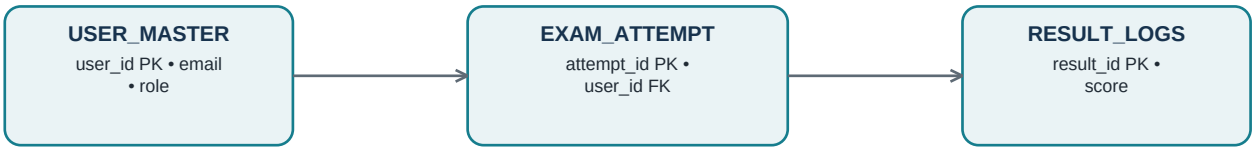
The main actors are Student and Administrator. Their privileges overlap only where necessary; question-bank maintenance is reserved for authorized administrators.

Actor	Use case	Precondition	Result
Student	Register	Registration form is available	Profile record created after validation
Student	Log in	Account exists and is active	Role-appropriate dashboard appears
Student	Attempt quiz	Authenticated; quiz is available	Answers collected and submitted
Student	View result	An eligible attempt exists	Own result displayed
Administrator	Create/edit question	Administrator session is valid	Question bank updated
Administrator	Review question bank	Administrator session is valid	Current questions listed
Administrator	Review results	Permission and policy allow access	Authorized summary displayed

Use-case acceptance depends on server-side authorization. Hiding an admin link in the browser is not sufficient protection if a student can call the endpoint directly.

APPENDIX D — CONCEPTUAL ENTITY RELATIONSHIP VIEW

The original report defines user and question tables. A complete quiz workflow generally also needs a record of each attempt and the result calculated for that attempt. The following conceptual view highlights these relationships; adapt the actual schema to the implemented code.



Conceptual relationship view; implementation may add answer-detail tables

Figure D.1. Simplified relationship view.

Entity	Primary key	Important attributes	Relationship
USER_MASTER	user_id	full_name, email_id, pass_hash, role_type	One user can make many attempts
QUESTION_BANK	question_id	subject_tag, query_text, options, correct_ans	Question can be included in many quizzes
EXAM_ATTEMPT	attempt_id	user_id, quiz_id, start_time, submit_time, status	Belongs to one user; may have one result
RESULT_LOGS	result_id	attempt_id, score, total_questions, evaluated_at	References a submitted attempt

For reliable auditing, store the final score against an attempt and avoid relying on a client-supplied correct answer. The authoritative answer key must be read from trusted server-side data.

APPENDIX E — DATABASE DATA DICTIONARY

The following tables are a recommended logical schema to accompany the concepts in the original report. The exact names and types should match the actual implementation before submission.

Table / field	Type	Key / constraint	Purpose
User_Master.user_id	INT	PK, auto increment	Unique user identifier
User_Master.full_name	VARCHAR(100)	NOT NULL	Display name
User_Master.email_id	VARCHAR(100)	UNIQUE, NOT NULL	Login identifier
User_Master.pass_hash	VARCHAR(255)	NOT NULL	Password hash, never plain text
User_Master.role_type	VARCHAR(20)	NOT NULL; allow-listed roles	Student or Admin
Question_Bank.question_id	INT	PK, auto increment	Unique question identifier
Question_Bank.query_text	TEXT	NOT NULL	Question statement
Question_Bank.correct_ans	CHAR(1)	Allow A/B/C/D	Authoritative answer key
Exam_Attempt.attempt_id	INT	PK, auto increment	Unique attempt
Result_Logs.result_id	INT	PK, auto increment	Unique result record

APPENDIX F — LOGIN VALIDATION FLOWCHART

The login process should validate the submitted credentials, retrieve the stored password hash, compare it using a suitable password-hashing library, and establish a session only after successful authentication. The role should be loaded from trusted database data.

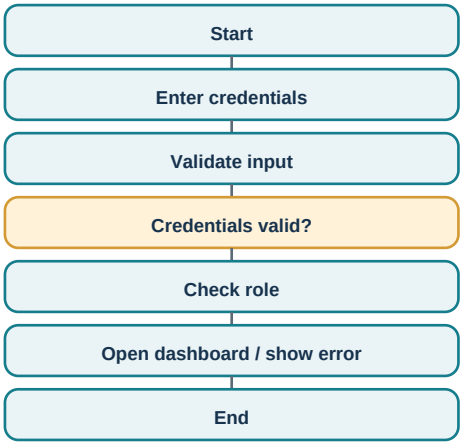


Figure F.1. Conceptual login validation flowchart.

Condition	Expected behavior
Required field missing	Show a general validation message; do not create a session
Unknown email or wrong password	Reject login without revealing which field was incorrect
Credentials valid	Regenerate session ID and load the stored role
Student role	Permit student routes only
Admin role	Permit admin routes only after authorization check
Unexpected database failure	Show a safe error message and log technical detail privately

APPENDIX G — QUIZ SUBMISSION FLOWCHART

A quiz attempt starts by loading the questions permitted for that assessment. The server accepts the student's selected options, checks the submission state, calculates the score against the stored answer key, and saves the result. Repeated submissions should be handled consistently to avoid duplicate scoring.

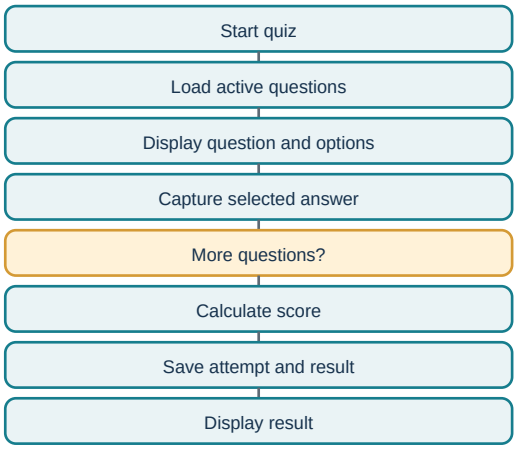


Figure G.1. Quiz attempt and scoring flowchart.

Recommended scoring rule: score = number of submitted answers that match the server-side correct answer. Missing answers should be treated according to the published quiz rules, normally as unanswered rather than as correct.

APPENDIX H — MODULE RESPONSIBILITIES

Module	Main responsibilities	Inputs / outputs	Validation focus
Authentication	Registration, login, session setup	Credentials/profile → dashboard or error	Unique email, password hash, session safety
Question management	Create, list, edit, delete questions	Question form → persisted question	Admin authorization, option completeness
Quiz delivery	Load available questions and accept choices	Quiz ID → question set	Availability, question identifiers, answer format
Evaluation	Compare selected options with answer key	Answer set → score	Use trusted answer key; handle missing answers
Result management	Save and retrieve attempt results	Attempt → result summary	Ownership, duplicate submissions, integrity
Error handling	Record faults and show safe messages	Exception → friendly error response	No passwords, secrets, or SQL in user-facing errors

Separating these responsibilities reduces controller complexity and makes unit testing easier. For a larger implementation, service classes and DAO/repository classes can isolate business rules from SQL details.

APPENDIX I — ALGORITHMS AND PSEUDOCODE

Algorithm 1: Objective quiz evaluation

Input: attempt identifier and submitted question-to-option selections.
Output: final score and saved result.

1. Confirm that the user is authenticated and allowed to submit this attempt.
2. Retrieve the authoritative question IDs and answer key from the database.
3. Initialize score = 0.
4. For each eligible question, read the submitted option.
5. If the option is valid and matches the stored answer key, increment score.
6. Save the attempt status and final score in a transaction.
7. Return the result summary to the authorized student.

Algorithm 2: Question creation

1. Verify the administrator role on the server.
2. Validate question text, all options, subject, and correct-answer value.
3. Normalize whitespace and enforce length limits.
4. Insert using a parameterized SQL statement.
5. Commit the transaction and show confirmation; otherwise log the error and show a safe failure message.

Complexity	Interpretation
Scoring: O(n)	Each of n questions is checked once, excluding database lookup costs
Question listing: O(n) transfer	Returned rows and page size affect response time
Indexing	Primary keys and unique email index help common lookups
Transactions	Keep attempt status and result record consistent

APPENDIX J — SECURITY AND PRIVACY CHECKLIST

Area	Recommended control	Verification evidence
Passwords	Use a modern adaptive password hash; never store plain text	Inspect stored values and authentication tests
SQL injection	Use PreparedStatement with bound parameters	Code review and hostile-input tests
Authorization	Check roles and record ownership on every protected route	Attempt direct URL access as student
Session	Use secure cookie settings, session timeout, and ID renewal after login	Inspect session behavior and cookie settings
CSRF	Use anti-CSRF tokens for state-changing requests	Submit a request without a valid token
Input validation	Validate length, type, allowed values, and identifiers server-side	Boundary and malformed-input tests
Error handling	Return generic errors; log details privately	Trigger a controlled database failure
Data protection	Limit access, back up data, define retention	Review permissions and backup procedure
Transport	Use HTTPS in deployed environments	Check deployment configuration

This checklist is a design recommendation, not a certification or evidence that every control has already been implemented. Record the actual test status before making a completion claim.

APPENDIX K — TEST PLAN AND SAMPLE TEST CASES

Test ID	Scenario	Test data / action	Expected result
T01	Valid login	Registered account and correct password	Correct role dashboard appears
T02	Invalid login	Wrong password	Access denied; no session established
T03	Duplicate registration	Email already in use	Clear validation message; no duplicate row
T04	Unauthorized admin URL	Student requests admin endpoint	Request denied on server
T05	Correct answer	Choose known correct option	Score increases by one
T06	Incorrect answer	Choose wrong option	Score does not increase
T07	Missing answer	Submit with one unanswered question	Handled according to quiz rules
T08	Repeated submission	Submit same attempt twice	No duplicate final result or score inflation
T09	Invalid question ID	Tamper with submitted identifier	Rejected or ignored safely
T10	Database unavailable	Stop database in test environment	Safe error shown; technical details logged
T11	Result ownership	Student requests another student's result	Access denied
T12	Responsive layout	Open page on narrow viewport	Controls remain readable and usable

Complete the Actual Result and Status columns during execution in the real environment. The expected outcomes above are test criteria, not claimed test results.

APPENDIX L — SAMPLE JDBC IMPLEMENTATION PATTERN

The code below is a compact pattern for a parameterized query. Configure the database URL and credentials outside source control, use a connection pool in larger deployments, and close JDBC resources reliably.

```
String sql = "SELECT user_id, pass_hash, role_type " + "FROM User_Master WHERE email_id = ?"; try (Connection con = dataSource.getConnection(); PreparedStatement ps = con.prepareStatement(sql)) { ps.setString(1, email); try (ResultSet rs = ps.executeQuery()) { if (rs.next()) { // Compare the submitted password with pass_hash // using a suitable password-hashing library. // Create a session only after successful comparison. } } } catch (SQLException ex) { // Log technical details on the server. // Return a generic message to the user. }
```

Implementation notes

- Do not compare a plain password directly with a stored hash.
- Do not accept a role supplied by the browser as authoritative.
- Use try-with-resources to close JDBC resources.
- Use a transaction when saving an attempt and its result together.
- Avoid printing stack traces into the browser response.

APPENDIX M — INTERFACE MOCK-UPS: LOGIN AND ADMIN

The images below are illustrative screen mock-ups created for report layout and design communication. They are not actual screenshots captured from a running project. Replace them with genuine application screenshots if the assessment requires proof of implementation.

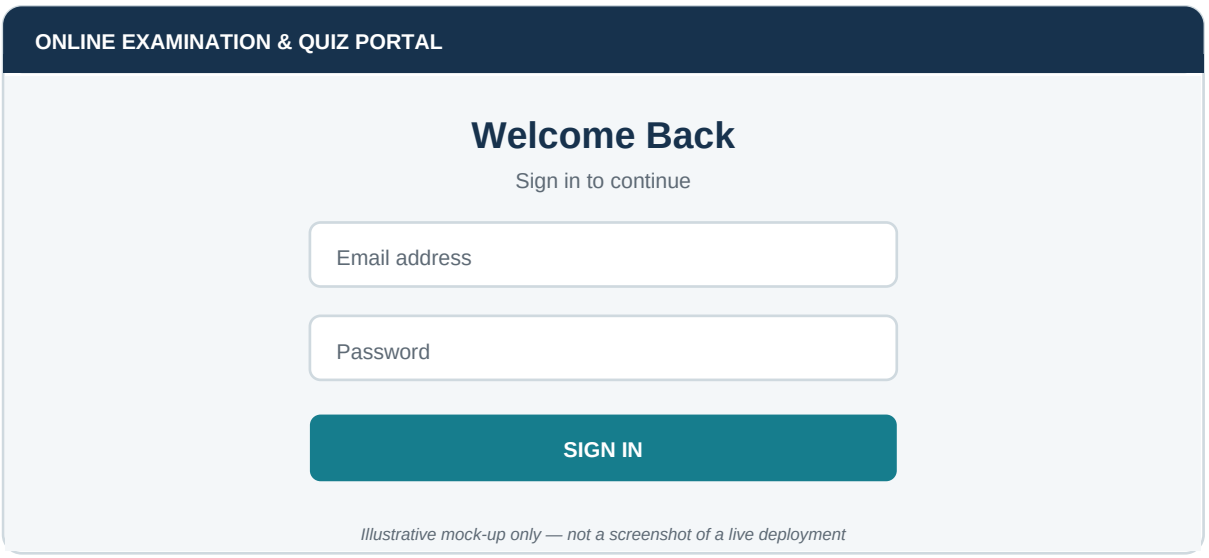


Figure M.1. Proposed student/admin login screen (illustrative).

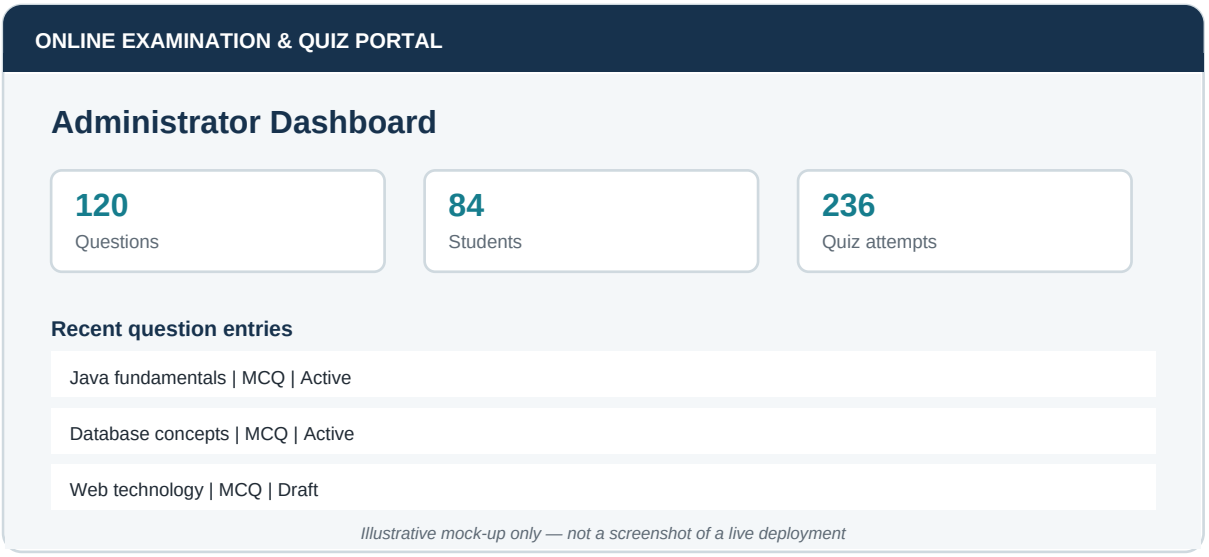


Figure M.2. Proposed administrator dashboard (illustrative).

Suggested real screenshot evidence: browser address bar or application title, visible page controls, and a representative test record. Remove personal data, passwords, session tokens, and database credentials before including screenshots.

APPENDIX N — INTERFACE MOCK-UPS: QUIZ AND RESULT

These mock-ups show the intended quiz and result presentation. Actual page appearance depends on the implemented CSS, data, and browser viewport.

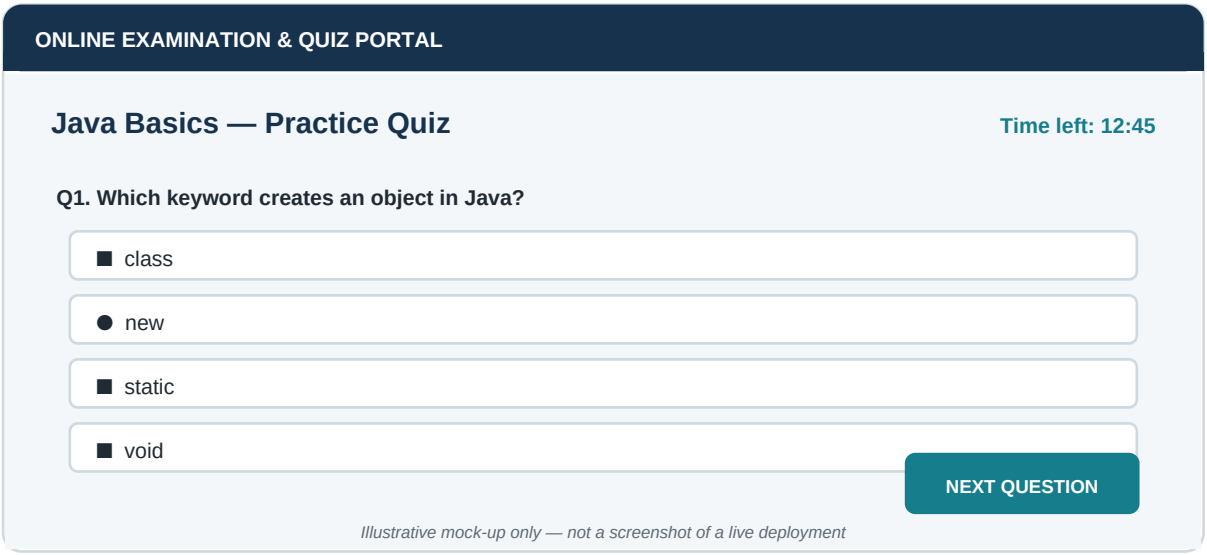


Figure N.1. Quiz question with multiple-choice options (illustrative).



Figure N.2. Result summary after submission (illustrative).

Recommended screenshots for a final submission: successful login, student dashboard, quiz question, validation error, result page, admin question list, and database rows showing a saved attempt with sensitive values hidden.

APPENDIX O — LIMITATIONS AND FUTURE ENHANCEMENTS

Area	Current documented scope	Possible enhancement
Question types	Objective multiple-choice questions	Add true/false, matching, and carefully designed subjective workflows
Assessment security	Basic authentication and role separation are proposed	Add stronger monitoring and institution-approved proctoring with privacy review
Analytics	Basic score display	Topic-level summaries and progress charts
Notifications	Not specified as implemented	Email notifications for schedules and result availability
Deployment	Local development stack described	Containerized deployment, HTTPS, monitoring, backups
Accessibility	Responsive web UI intended	Formal keyboard, screen-reader, contrast, and accessibility testing
Scalability	Single application/database design	Load testing, connection pooling, caching, and service scaling

Enhancements should be prioritized based on user needs, available resources, data protection obligations, and evidence from testing. Features such as facial recognition require careful legal, ethical, and privacy assessment before adoption.

Conclusion

The Online Examination and Quiz Portal illustrates how Java web technologies and a relational database can support a structured objective-assessment workflow. A sound architecture, explicit requirements, secure implementation practices, and documented testing are essential to moving from a prototype to a reliable service.

APPENDIX P — REFERENCES AND FINAL SUBMISSION CHECKLIST

References

1. Schildt, H. (2018). *Java: The Complete Reference* (11th ed.). McGraw-Hill Education.
2. Basham, B., Sierra, K., & Bates, B. (2008). *Head First Servlets and JSP*. O'Reilly Media.
3. Elmasri, R., & Navathe, S. B. (2017). *Fundamentals of Database Systems* (7th ed.). Pearson.
4. Oracle. *Java Servlet Technology and Java Platform Documentation*. Consult the official documentation for the versions used.
5. Apache Software Foundation. *Apache Tomcat Documentation*. Consult the documentation matching the installed Tomcat version.
6. Oracle. *JDBC Basics*. Consult the official Java database connectivity documentation.
7. MySQL. *MySQL Reference Manual*. Consult the version-specific manual for SQL syntax and constraints.

Before printing and submission

- ■ Confirm the department's required cover-page wording and logo.
- ■ Verify student name, register number, degree, semester, academic year, and organisation details.
- ■ Obtain required signatures and certificate approvals.
- ■ Replace illustrative mock-ups with real screenshots if required; label every image accurately.
- ■ Update the table of contents and page references if institutional formatting changes.
- ■ Run the test cases and record actual outcomes rather than assuming success.
- ■ Check spelling, page order, margins, and readability in a print preview.
- ■ Confirm that confidential credentials and personal information are not present.

End of supplementary documentation.